

Number and Group Theory

TC50331E

Algebraic Coding Theory

Jaish Singh
33139652

April 2026

Every time data travels across a network, a file being downloaded or a being message sent, there is a chance that some bits flip to the opposite sign. The challenge of reliable communication is therefore not only one of engineering, but of mathematics. How do we design systems that can detect and correct errors, without knowing in advance where they will occur? Algebraic coding theory answers this question by imposing carefully chosen mathematical structure on transmitted data. This report develops the theory of binary block codes, showing how group theory provides the natural language for describing and detecting errors.

1 Block Codes

Let $\mathbf{B}(n) = \underbrace{\mathbb{Z}_2 \times \mathbb{Z}_2 \times \cdots \times \mathbb{Z}_2}_{n \text{ copies}}$, the set of all binary strings of length n . That is,

$$\mathbf{B}(n) = \{(x_1, \dots, x_n) \mid x_i \in \mathbb{Z}_2 \text{ for } 1 \leq i \leq n\}.$$

We equip this set with a binary component-wise addition in modulo 2:

$$(x_1, \dots, x_n) + (y_1, \dots, y_n) = (x_1 + y_1, \dots, x_n + y_n)$$

where addition is taken in $\mathbb{Z}_2$. This operation is well-defined since $\mathbb{Z}_2$ is closed under addition. We verify the group axioms for $(\mathbf{B}(n), +)$:

1. *Closure:* Let $x, y \in \mathbf{B}(n)$. Then $x + y \in \mathbf{B}(n)$ since addition in $\mathbb{Z}_2$ is closed.
2. *Associativity:* For all $x, y, z \in \mathbf{B}(n)$, we have

$$(x + y) + z = x + (y + z),$$

since addition in $\mathbb{Z}_2$ is associative.

3. *Identity:* The element $0 = (0, \dots, 0) \in \mathbf{B}(n)$ satisfies $x + 0 = x$ for all $x \in \mathbf{B}(n)$.
4. *Inverses:* For each $x \in \mathbf{B}(n)$, we have $x + x = 0$ since $0 + 0 = 0$ and $1 + 1 = 0$, so every element is its own inverse.

Having established that $(\mathbf{B}(n), +)$ is a group, we observe that it is in fact Abelian. This follows from the commutativity of addition in $\mathbb{Z}_2$, applied component-wise. Since digital signals are inherently binary, it is natural to model transmitted data as elements of $\mathbb{Z}_2 = \{0, 1\}$, the group of integers modulo 2 under addition. The group operation, which coincides with XOR, captures

parity computations precisely: the parity of a binary string is simply its sum in $\mathbb{Z}_2$. This structure extends naturally to $\mathbb{Z}_2^n$, the group of binary strings of length n , which serves as the space in which code words live.

For $0 < k < n$, we define an (n, k) block code to be a subset of $\mathbf{B}(n)$, such that the cardinality of the block code $(n, k) = 2^k$, and elements are binary strings, called code words, of length n . We restrict transmission of binary data to only the code words within the prescribed block code. Since bit flips can happen seemingly at random, the code words may get transformed to binary strings outside the block code. Hence, the received words are elements of a broader set of all binary strings of length n , $\mathbf{B}(n)$. Additionally, if a block code happens to be a subgroup of $\mathbf{B}(n)$, then this is a special type of block code called a Linear Block Code. Linear Block Codes have several structural qualities, explored in more depth in the subsequent sections, that make them more efficient.

For instance, consider a set $C = \{(0, 0, 0, 0), (0, 1, 0, 1), (1, 0, 1, 0), (1, 1, 1, 1)\}$. C is a $(4, 2)$ block code since every code word is of length 4 and $|C| = 4 = 2^2$ corresponding to $k = 2$ information bits. Since transmission may cause errors anywhere in the string, the received words for any element in C are all binary strings of length 4, i.e. any element in $\mathbf{B}(4) = \{0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111\}$. For $a, b \in C$, $a + b^{-1} \in C$, which establishes that C is a subgroup of $\mathbf{B}(4)$, and therefore C is a Linear Block Code.

A generator matrix $\mathbf{G}$, for a (n, k) block code C is a $k \times n$ matrix whose rows form a basis for C , i.e. are linearly independent. We use a generator matrix [1] to systematically generate code words in a block code. For instance, the generator matrix $\mathbf{G}$, for a $(5, 2)$ block code $C = \{(0, 0, 0, 0, 0), (1, 0, 1, 1, 0), (0, 1, 1, 0, 1), (1, 1, 0, 1, 1)\}$ is defined as

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

To verify if a received word is a valid code word, we make use of a $(n - k) \times n$ parity-check matrix, H . The valid code words are the null space of the parity-check matrix, i.e.,

$$HG^T = 0$$

Similarly, a received word C is a valid code word if it satisfies the equation $HC^T = 0$. The parity-check matrix, H , for our $(5, 2)$ block code is

$$H = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}$$

$$HG^T = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

For a given block code, we must define its capabilities in terms of error detection and error correction. This capability is defined in terms of the *weight* of a code word in a block code. The weight of a binary string $b \in \mathbf{B}(n)$, denoted by $w(b)$, is the count of 1s in b . For instance, the weight of $(1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1) \in \mathbf{B}(12)$ is 8. The *distance* d , between two binary strings, $x, y \in \mathbf{B}(n)$, is the weight of their sum in $\mathbb{Z}_2$, i.e.,

$$d(x, y) = w(x + y)$$

In other words, the distance is the number of positions where the two strings differ. The distance defines a metric [3] on $\mathbf{B}(n)$. Formally, a metric defined on a set is a function such that it follows

1. Definiteness: $d(x, y) = 0 \iff x = y$
2. Symmetry: $d(x, y) = d(y, x)$
3. Triangle inequality: $d(x, z) \leq d(x, y) + d(y, z)$

It is straightforward to verify that d satisfies each of these.

1. Definiteness: $d(x, y) = w(x + y) = 0$ if and only if $x + y = 0$, which holds if and only if $x_i = y_i$ for all i , i.e. $x = y$.
2. Symmetry: $d(x, y) = w(x + y) = w(y + x) = d(y, x)$, since $x_i + y_i = y_i + x_i$ for each bit.
3. Triangle inequality: For any $x, y, z \in \mathbf{B}(n)$, note that

$$\begin{aligned}
 (x + z) &= x + 0 + z \\
 &= x + (y + y) + z \\
 &= (x + y) + (y + z)
 \end{aligned}$$

so,

$$\begin{aligned}
 d(x, z) &= w(x + z) \\
 &= w((x + y) + (y + z)) \leq w(x + y) + w(y + z) \\
 &= d(x, y) + d(y, z)
 \end{aligned}$$

where the inequality uses subadditivity of weight: $(a+b)_i = 1$ only if $a_i = 1$ or $b_i = 1$, so every position counted in $w(a + b)$ is also counted in $w(a) + w(b)$, giving $w(a + b) \leq w(a) + w(b)$.

This metric structure carries directly into the analysis of block codes, where distance between codewords becomes the central quantity of interest.

2 Decoding and Errors

Let C be a code word to be transmitted, and R its received counterpart. The error in transmission is measured by the distance between the sent binary string and the received string, $d(C, R)$. These errors change the message seemingly at random coordinates, and therefore, there is a non zero chance for the received message to have been transformed to another valid code word. If C is corrupted by e errors, R lies at a distance e from C . Let $d_{\min}$ be the smallest distance between any two valid code words in a block code. Provided $e < d_{\min}$, the received word cannot coincide with any other valid codeword. Therefore the decoder can detect up to $d_{\min} - 1$ errors. If R is detected to have errors, it may be possible to error correct the message. We simply do this by taking the nearest valid code word to R as the intended message.

Correction is possible only when R has a unique nearest codeword, i.e., it is not equidistant between two codewords. Conversely, R must be at most $\lfloor \frac{d_{\min}-1}{2} \rfloor$ from a valid code word, where $\lfloor \cdot \rfloor$ denotes the floor function, rounding down to the nearest integer. Therefore, a code with minimum distance $d_{\min}$ can correct at most $\lfloor \frac{d_{\min}-1}{2} \rfloor$ errors, since any R within that radius is guaranteed to be closer to exactly one valid code word than any other. In other words, $d_{\min} \geq 2t + 1$ [2], where t is the number of errors the block code can correct. Conversely, suppose for contradiction that a block code $\mathbf{C}$ corrects t errors but $d_{\min} \leq 2t$. Then there exist distinct codewords $C_1, C_2 \in \mathbf{C}$ with $d(C_1, C_2) \leq 2t$. We construct R by flipping $\lfloor \frac{d(C_1, C_2)}{2} \rfloor$ bits of C_1 toward C_2 . Then $d(R, C_1) \leq t$

and $d(R, C_2) \leq t$, so nearest-neighbour decoding cannot uniquely recover the original code word. This contradicts our assumption, therefore we establish that a block code can recover t errors if and only if $d_{\min} \geq 2t + 1$.

Let $\mathbf{G}$ be a block code described by

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

where the code words for $\mathbf{G}$ are defined by the encoding algorithm

$$\begin{aligned} \mathbf{C} &= \{\mathbf{B}\mathbf{G} : B \in \mathbf{B}(3)\} \subset \mathbf{B}(6) \\ &= \{000000, 100111, 010101, 001110, 110010, 101001, 011011, 111100\} \end{aligned}$$

Since $\mathbf{C}$ is closed under addition in $\mathbb{Z}_2$ and contains 000000, $\mathbf{C}$ is a group and consequently a Linear Block Code. As a result, for any two code words $C_1, C_2 \in \mathbf{C}$, their sum $C_1 + C_2 \in \mathbf{C}$, and the distance satisfies $d(C_1, C_2) = w(C_1 + C_2)$. Therefore $d_{\min}$, the smallest distance between any two valid code words, would be the smallest weight of a codeword in $\mathbf{C}$ excluding 000000, which by inspection we deduce to be 3, the weight of 010101. Using $d_{\min}$, we infer that $\mathbf{G}$ can detect at most $d_{\min} - 1 = 2$ errors and correct $\lfloor \frac{d_{\min}-1}{2} \rfloor = 1$ error.

The parity-check matrix, $\mathbf{H}$, validates any input $B \in \mathbf{B}(n)$ given by $F : \mathbf{B}(n) \rightarrow \mathbf{B}(n - k)$ such that,

$$F(B) = \mathbf{H}B^t, B \in \mathbf{B}(n)$$

If an error is detected, we take the nearest valid code word to be the message. However, to search for the nearest valid code word by individually measuring the distance between every possible word would be computationally expensive. To get around this, we use the fact that F is a surjective group homomorphism. Since F satisfies the condition $F(a + b) = F(a) + F(b)$ which follows from the linearity of matrix multiplication, F is a group homomorphism. We also know that $\mathbf{H}$ has a full row rank $n - k$ which is an inherent property of $\mathbf{H}$ by the definition of a parity-check matrix. This suggests that the linear map F can produce any vector in $\mathbf{B}(n - k)$ and therefore is a surjective map. Thus, we deduce that F is a surjective group homomorphism, which allows us to partition $\mathbf{B}(n)$ into cosets of the kernel of F .

$$F(B) = \mathbf{H}B^t = \mathbf{H}(C + e)^t = \mathbf{H}C^t + \mathbf{H}e^t = 0 + \mathbf{H}e^t = \mathbf{H}e^t$$

where e is the error in B . We precompute all cosets of the kernel of F in $\mathbf{B}(n)$ and find the element with the smallest weight in each coset, called the coset leader, $\hat{e}$. When decoding, we find the coset in which $F(B) = \mathbf{H}e^t$ belongs to in our precomputed set of cosets. Once we have found our coset, and its corresponding coset leader $\hat{e}$, finding the closest code word $\hat{C}$ becomes trivial

$$\hat{C} = B + \hat{e}$$

Since F is a group homomorphism, its kernel $\ker(F) = \{B \in \mathbf{B}(n) : \mathbf{H}B^t = 0\}$ is a subgroup of $\mathbf{B}(n)$. Because $\mathbf{H}\mathbf{G}^t = 0$, every codeword $C = \mathbf{B}\mathbf{G}$ satisfies $F(C) = \mathbf{H}C^t = 0$, so all valid codewords lie in $\ker(F)$. Conversely, any $B \in \ker(F)$ satisfies $\mathbf{H}B^t = 0$ and therefore belongs to the block code. Hence $B \in \mathbf{B}(n)$ is a codeword if and only if $\mathbf{H}B^t = 0$.

A natural follow up to this is understanding how efficiently we can encode information, i.e., given a block length n , how many information bits k can we transmit while still correcting t errors? Since $|\mathbf{C}| = 2^k$, bounding the number of codewords directly bounds k . Each codeword C occupies a sphere of radius t , containing $\sum_{i=0}^t \binom{n}{i}$ elements of $\mathbf{B}(n)$. Since these spheres must be disjoint, $|\mathbf{C}| \cdot \sum_{i=0}^t \binom{n}{i} \leq 2^n$. For $t = 1$ this gives $2^k(1 + n) \leq 2^n$, or equivalently $k \leq n - \log_2(1 + n)$ [1]. A Hamming code is a linear block code that achieves exactly this bound. Therefore every element of $\mathbf{B}(n)$ lies in exactly one decoding sphere. Formally, a Hamming code with redundancy m has parameters $n = 2^m - 1$, $k = 2^m - 1 - m$, with the parity-check matrix H having dimensions $m \times (2^m - 1)$. Since no column of H is zero and no two columns are the same, no codeword of weight 1 or 2 can exist (since $\mathbf{H}B^t = 0$ would require a 0 column or two identical columns).

Thus, $d_{\min} \geq 3$. Since three columns can sum to zero (for instance, any column h_i satisfies $h_i + h_j + (h_i + h_j) = 0$ for distinct i, j , and $h_i + h_j$ is itself a non-zero column of H), some weight-3 codeword exists, giving $d_{\min} = 3$ exactly [3].

A received word B is a valid codeword if and only if $HB^T = 0$. If a single-bit error occurs at position i , the syndrome HB^T equals precisely the i^{th} column of H , which uniquely identifies the coset containing B . Since each single-bit error vector is the coset leader of a distinct coset, guaranteed by the fact that all columns of H are distinct and non-zero, we can recover $\hat{e}$ directly from the syndrome, making the lookup into our precomputed coset table trivial.

3 Empirical Investigation

3.1 Setup

The simulation [4] consists of a `Block_code` class that takes codewords $\mathbf{C} \subseteq \mathbf{B}(n)$ as input. From the list of code words, we determine $k = \log_2 |\mathbf{C}|$. We assume the input to be valid and a linear block code and therefore $\mathbf{C}$ to be a subgroup of $\mathbf{B}(n)$. Since we are dealing with linear block codes, the distance between any two codewords $C_1, C_2 \in \mathbf{C}$ satisfies $d(C_1, C_2) = w(C_1 + C_2) \in \mathbf{C}$. We use this to compute $d_{\min}$ by calculating the weight of each code word. The error-detection and correction capacities are then computed as $d_{\min} - 1$ and $\lfloor \frac{d_{\min}-1}{2} \rfloor$, respectively.

Encoding is described by the matrix product $C = \mathbf{m} \cdot G$. Here, $\mathbf{m}$ is a list of length k containing the message in binary format. C is the resulting codeword of length n , and G is the $k \times n$ generator matrix derived from $\mathbf{C}$ via Gaussian elimination. `compute_generator_matrix` row-reduces the entire list of codewords to identify k linearly independent rows, resulting in G in systematic form $[I_k \mid P]$. The parity-check matrix is then computed algebraically as $H = [P^T \mid I_{n-k}]$, which follows $HG^T = 0$ by design.

When initializing the block code, we run `compute_syndrome_table` to systematically compute all error patterns and their corresponding syndrome in order of increasing weight. The first error pattern computed for each syndrome is recorded as the coset leader $\hat{e}$. The table is complete when it has 2^{n-k} entries, since $\mathbf{B}(n)$ divides into exactly 2^{n-k} cosets of $\mathbf{C}$, as shown by the surjectivity of $F(B) = HB^T$. During runtime, `correct` computes the syndrome of the received word with simulated random errors. By matching the syndrome with the coset leaders in the syndrome table, we find the corresponding error pattern. We then add this error pattern to R since the inverse of the error is itself. This subsequently outputs the corrected codeword ($\hat{C} = R + \hat{e}$). If the weight of the coset leader is larger than $\lfloor \frac{d_{\min}-1}{2} \rfloor$, an error is raised because reliable correction is not guaranteed. Note that while generating the syndrome table is expensive since we compute every error pattern possible for the block code, this is only done once at initialization which allows us to have very inexpensive correction of received words at runtime.

We define `send`, to randomly add error to a code word. `send` takes a codeword and an error count. It randomly picks positions in the binary string to flip to the opposite sign. We simulate the transmission scenario described in Section 2. A codeword $\mathbf{C}$ is sent, random errors occur, and a received word R comes back with simulated errors. We use this to observe characteristics of block codes empirically.

One limitation of this implementation is that `compute_syndrome_table` has a memory cost of $O(2^{n-k})$, which can become problematic for large n . Additionally, `send` assumes errors happen independently at random positions, which is an idealized assumption. In reality, errors may cluster together, affecting more positions than the model expects.

3.2 Experiment

We systematically vary n and k for a block code to observe changes in $d_{\min}$. We use $d_{\min}$ as a proxy for our block codes' utility since both error correction as well as error detection capacity of a block code are computed using $d_{\min}$, where larger $d_{\min}$ relates to better error-control capabilities.

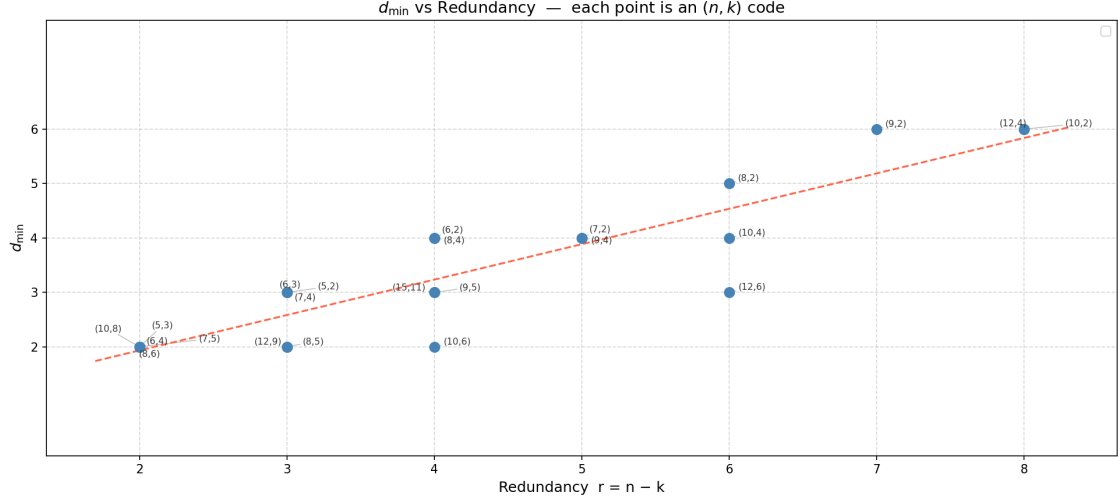


Figure 1: $d_{\min}$ vs Redundancy for varying block codes

The scatter plot confirms a positive trend between redundancy $r = n - k$ and $d_{\min}$: as more redundant bits are added, larger $d_{\min}$ becomes achievable. However, the vertical scatter at each value of r suggests that while redundancy is a necessary condition for $d_{\min}$, it does not fully explain $d_{\min}$. At $r = 4$, $d_{\min}$ varies from 2 for the (10,6) block code to 4 for (6,2). The (15,11) Hamming code is a significant point of interest: it attains $d_{\min} = 3$ at $r = 4$ while encoding 11 information bits. This highlights that the structure of a block code is a significant factor when assessing its utility. The dashed trend line captures the general correlation between $d_{\min}$ and r , but the scatter around it emphasizes that choosing a block code requires optimising construction, not just increasing redundancy.

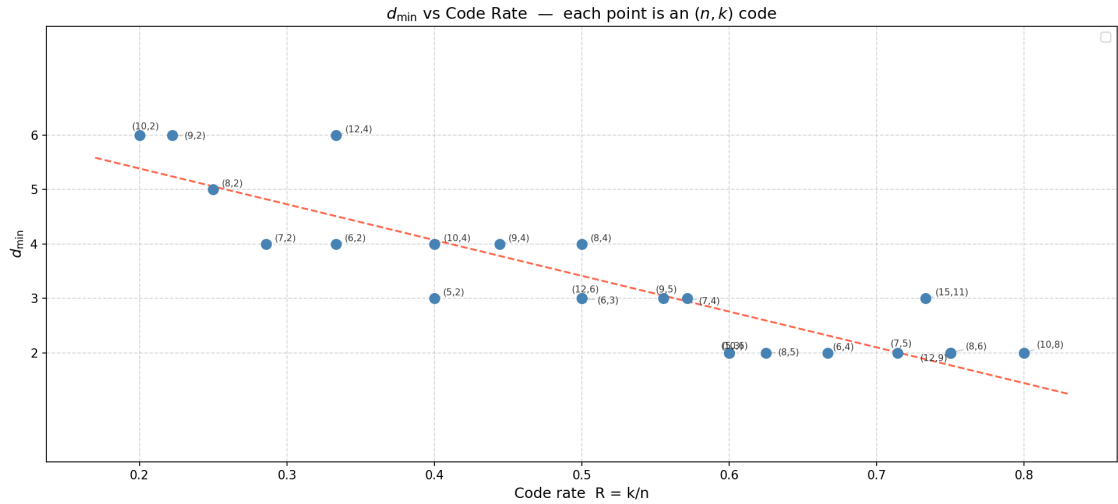


Figure 2: $d_{\min}$ vs Code Rate for varying block codes [Table 1]

Figure 2 confirms the underlying antagonism between code rate $R = k/n$ and error-correction capacity. As R increases, the achievable $d_{\min}$ falls. The negative trend is prominent, but the scatter around it again shows that rate alone does not fully explain $d_{\min}$. We notice that the (12,4)

block code is a significant outlier, attaining $d_{\min} = 6$ at $R \approx 0.33$, well above comparable code rates, highlighting how longer block lengths can achieve greater distance at the same rate. At the high-rate end, the (15,11) Hamming code stands out for attaining $d_{\min} = 3$ at $R \approx 0.73$, an area where most block codes max out at $d_{\min} = 2$. A noticeable floor at $d_{\min} = 2$ appears for $R \geq 0.6$, suggesting that beyond this threshold, further increases in code rate do not meaningfully worsen $d_{\min}$. This indicates that $d_{\min}$ is already at its practical minimum. The widest vertical scatter appears at low R , indicating that block codes with low R offer the greatest design flexibility, while block codes with high R converge toward the minimum distance floor.

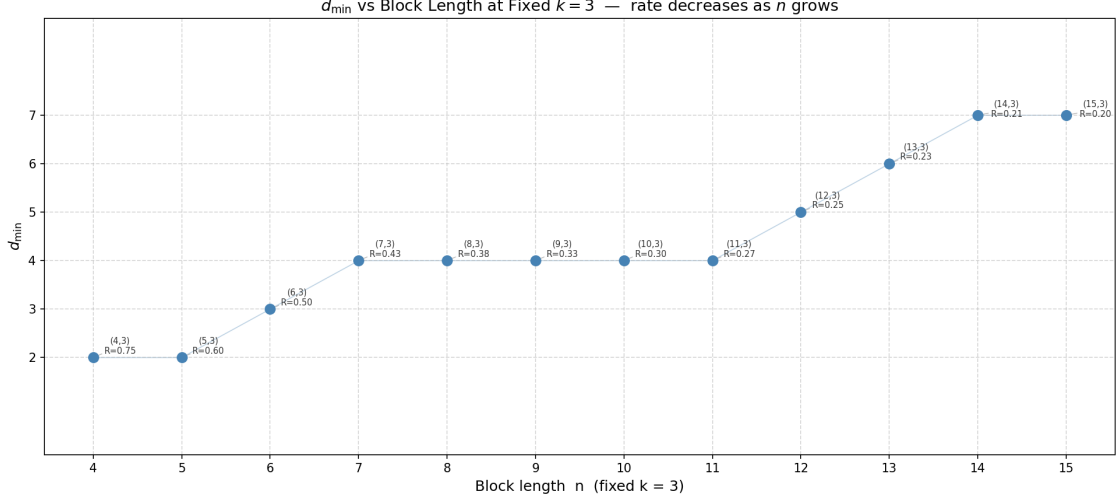


Figure 3: $d_{\min}$ vs n for fixed k [Table 2]

With k fixed at 3, increasing the block length n is equivalent to adding redundancy while holding information bits constant, and Figure 3 shows that this reliably grows $d_{\min}$, but not linearly. The progression is distinctly step-wise, $d_{\min}$ increases are separated by flat plateaus, reflecting the discrete geometry of Hamming space rather than a continuous trade-off. The most notable plateau spans from $n = 7$ to $n = 11$, where five block codes share $d_{\min} = 4$ even as the R falls from 0.43 to 0.27, four extra redundant bits that result in no improvement in $d_{\min}$. The escape from the plateau at $n = 12$ suggests that increases in distance are unlocked only at specific thresholds, making naive redundancy addition an inefficient design strategy.

Figures 1-3 give a coherent and practically important picture of block code design. Figure 1 establishes that redundancy is a prerequisite for large $d_{\min}$, but construction quality determines how efficiently that redundancy is converted. Figure 2 reframes the same trade-off through code rate, revealing a $d_{\min}$ floor at high rates and the greatest design freedom at low rates. Figure 3 isolates block length, even with k fixed, increases in $d_{\min}$ are not proportional to added redundancy. They arrive in discrete steps, with long plateaus where extra bits are effectively wasted. Across all three perspectives, the conclusion is the same, parameter choices set the ceiling on achievable $d_{\min}$, but it is the algebraic structure of the code that determines whether that ceiling is approached. Efficient block code design is therefore not just a matter of allocating more bits to redundancy, but of constructing codes whose geometry exploits every redundant bit to maximally separate codewords.

References

- [1] ER Berlekamp. “A survey of coding theory”. In: *Journal of the Royal Statistical Society Series A: Statistics in Society* 135.1 (1972), pp. 44–73.
- [2] Raj Chandra Bose and Dwijendra K Ray-Chaudhuri. “On a class of error correcting binary group codes”. In: *Information and control* 3.1 (1960), pp. 68–79.
- [3] Richard W Hamming. “Error detecting and error correcting codes”. In: *The Bell system technical journal* 29.2 (1950), pp. 147–160.
- [4] Jaish Singh. *Algebraic Coding Theory, Coursework*. 2026. URL: https://github.com/jbsx/group_theory_coursework (accessed on 18/5/2026).

A Appendix

n	k	Redundancy ($n - k$)	code rate (k/n)	$d_{\min}$
5	2	3	0.40	3
5	3	2	0.60	2
6	2	4	0.33	4
6	3	3	0.50	3
6	4	2	0.67	2
7	2	5	0.29	4
7	4	3	0.57	3
7	5	2	0.71	2
8	2	6	0.25	5
8	4	4	0.50	4
8	5	3	0.63	2
8	6	2	0.75	2
9	2	7	0.22	6
9	4	5	0.44	4
9	5	4	0.56	3
10	2	8	0.20	6
10	4	6	0.40	4
10	6	4	0.60	2
10	8	2	0.80	2
12	4	8	0.33	6
12	6	6	0.50	3
12	9	3	0.75	2
15	11	4	0.73	3

Table 1: $d_{\min}$ for block codes of varying n and k [4]

n	k	Redundancy ($n - k$)	code rate (k/n)	$d_{\min}$
4	3	1	0.75	2
5	3	2	0.60	2
6	3	3	0.50	3
7	3	4	0.43	4
8	3	5	0.38	4
9	3	6	0.33	4
10	3	7	0.30	4
11	3	8	0.27	4
12	3	9	0.25	5
13	3	10	0.23	6
14	3	11	0.21	7
15	3	12	0.20	7

Table 2: $d_{\min}$ for fixed $k = 3$ and varying n [4]